

Variables & JavaScript Data Types

Section 1 — JavaScript Fundamentals

JavaScript শেখার শুরুতেই সবচেয়ে গুরুত্বপূর্ণ দুইটা জিনিস হলো:

1. কীভাবে **data store** করব? → Variables
2. কী ধরনের **data store** করছি? → Data Types

এই দুইটা concept ভালোভাবে বুঝলে JavaScript-এর পরের অনেক topic—functions, objects, arrays, conditions, DOM, React state—অনেক সহজ হয়ে যাবে।

1. What Is a Variable?

একটি **variable** হলো এমন একটি named container/reference যেখানে আমরা কোনো value রাখি, যাতে পরে সেই value ব্যবহার করতে পারি।

সহজভাবে:

```
Variable
  ↓
একটি নাম
  ↓
একটি value-এর সাথে সম্পর্ক
```

Example:

```
let age = 25;
```

এখানে:

- `let` → variable declaration keyword
- `age` → variable name
- `25` → value

পরে আমরা `age` ব্যবহার করতে পারি:

```
console.log(age);
```

Output:

2. Variable কেন দরকার?

ধরো তোমার application-এ user-এর নাম বারবার দরকার:

```
console.log("Shariar");  
console.log("Welcome Shariar");  
console.log("Hello Shariar");
```

এভাবে hard-code করলে code maintain করা কঠিন।

Variable ব্যবহার করলে:

```
const userName = "Shariar";  
  
console.log(userName);  
console.log(`Welcome ${userName}`);  
console.log(`Hello ${userName}`);
```

এখন user-এর নাম পরিবর্তন করতে হলে এক জায়গাতেই পরিবর্তন করতে হবে।

```
const userName = "Duke";
```

3. Variable Declaration

JavaScript-এ variable declare করার প্রধান তিনটি keyword:

```
var  
let  
const
```

Example:

```
var name = "Shariar";

let age = 31;

const country = "Bangladesh";
```

তিনটিই variable তৈরি করতে পারে, কিন্তু তাদের behavior এক নয়।

Modern JavaScript-এ সাধারণত:

```
const → default choice
let   → value পরিবর্তন হলে
var   → সাধারণত avoid
```

4. **let**

let ব্যবহার করে এমন variable তৈরি করা হয় যার value পরে পরিবর্তন হতে পারে।

```
let age = 25;

age = 26;

console.log(age);
```

Output:

```
26
```

এখানে প্রথমে:

```
let age = 25;
```

তারপর:

```
age = 26;
```

করা হয়েছে।

এটাকে বলা হয় **reassignment**।

4.1 `let` Reassignment

```
let score = 50;

score = 80;

score = 95;

console.log(score);
```

Output:

```
95
```

একটি `let` variable-এ নতুন value assign করা যায়।

4.2 `let` Redeclaration

একই scope-এর মধ্যে একই নাম আবার declare করা যায় না।

```
let age = 25;

let age = 30;
```

এতে error হবে:

```
SyntaxError
```

তবে value পরিবর্তন করা যাবে:

```
let age = 25;

age = 30;
```

এটা valid।

5. `const`

`const` ব্যবহার করা হয় এমন variable-এর জন্য যেটাকে **reassign** করা হবে না।

```
const country = "Bangladesh";

console.log(country);
```

এখন:

```
country = "India";
```

করলে error হবে।

কারণ `const` variable-কে নতুন value assign করা যায় না।

5.1 `const` Must Be Initialized

`const` declare করার সময় value দিতে হয়।

❌ Invalid:

```
const age;
```

কারণ `const` declaration-এর সাথে initialization থাকতে হবে।

✅ Valid:

```
const age = 31;
```

6. `var`

`var` হলো JavaScript-এর পুরোনো variable declaration system।

```
var name = "Shariar";

name = "Duke";

console.log(name);
```

এটা valid।

`var` -এর সবচেয়ে গুরুত্বপূর্ণ সমস্যা হলো এর **function scope** এবং **redeclaration behavior**।

6.1 `var` Redeclaration

`var` একই scope-এ আবার declare করা যায়।

```
var age = 25;

var age = 30;

console.log(age);
```

Output:

```
30
```

এটা `let` এবং `const` থেকে আলাদা।

7. `var` vs `let` vs `const`

Feature	<code>var</code>	<code>let</code>	<code>const</code>
Reassign করা যায়	✓	✓	✗
Redeclare করা যায়	✓	✗	✗
Block scoped	✗	✓	✓
Function scoped	✓	No	No
Hoisted	✓	✓	✓
TDZ	✗	✓	✓
Modern JS-এ recommended	সাধারণত ✗	✓	✓

8. Scope

Scope বলতে বোঝায়—কোন জায়গা থেকে একটি variable access করা যাবে।

JavaScript-এ গুরুত্বপূর্ণ scope:

- Global Scope
- Function Scope
- Block Scope

9. Global Scope

যদি variable কোনো function বা block-এর বাইরে declare করা হয়, সেটি সাধারণত global scope-এ থাকে।

```
const name = "Shariar";

function greet() {
  console.log(name);
}

greet();
```

Output:

```
Shariar
```

Function-এর ভিতর থেকে বাইরের variable access করা যাচ্ছে।

10. Function Scope

Function-এর ভিতরে declare করা variable function-এর বাইরে access করা যায় না।

```
function test() {
  var message = "Hello";

  console.log(message);
}

test();

console.log(message);
```

শেষের:

```
console.log(message);
```

error দেবে।

কারণ `message` function scope-এর ভিতরে।

11. Block Scope

Block বলতে সাধারণত `{ }` দিয়ে তৈরি অংশকে বোঝায়।

যেমন:

```
if (true) {  
  // block  
}
```

`let` এবং `const` block scoped।

```
if (true) {  
  let age = 31;  
  const name = "Shariar";  
  
  console.log(age);  
  console.log(name);  
}
```

Block-এর ভিতরে access করা যাবে।

কিন্তু:

```
if (true) {  
  let age = 31;  
}  
  
console.log(age);
```

error হবে।

কারণ `age` block-এর বাইরে accessible নয়।

12. `var` vs Block Scope

এখানেই `var` এবং `let` / `const` -এর একটি গুরুত্বপূর্ণ পার্থক্য।

```
if (true) {  
    var name = "Shariar";  
}  
  
console.log(name);
```

Output:

```
Shariar
```

কারণ `var` block scoped নয়।

কিন্তু:

```
if (true) {  
    let name = "Shariar";  
}  
  
console.log(name);
```

এখানে error হবে।

13. কেন Modern JavaScript-এ `var` Avoid করা হয়?

`var` নিজে "broken" না। কিন্তু এর পুরোনো scoping behavior অনেক confusing situation তৈরি করতে পারে। বিশেষ করে:

- function scope
- block scope না থাকা
- redeclaration
- hoisting behavior

এসব কারণে modern JavaScript code-এ সাধারণত:

```
const  
let
```

ব্যবহার করা হয়।

Practical Rule

প্রথমে ভাবো:

এই *variable*-এর *value* কি পরিবর্তন হবে?

যদি না হয়:

```
const
```

যদি পরিবর্তন হয়:

```
let
```

`var` সাধারণত নতুন code-এ ব্যবহার করার প্রয়োজন হয় না।

14. Declaration vs Initialization vs Assignment

এই তিনটি term খুব ভালোভাবে বুঝে রাখো।

Declaration

```
let age;
```

এখানে variable declare করা হয়েছে।

Initialization

```
let age = 31;
```

Declaration-এর সময় প্রথম value দেওয়া হচ্ছে।

Assignment / Reassignment

```
age = 35;
```

আগের value পরিবর্তন করা হচ্ছে।

15. JavaScript Data Types

এখন আসি দ্বিতীয় গুরুত্বপূর্ণ concept-এ:

Data Type

Data type বলে দেয় একটি value কী ধরনের data represent করছে।

উদাহরণ:

```
"Shariar"
```

এটি string।

```
31
```

এটি number।

```
true
```

এটি boolean।

JavaScript-এর প্রধান data types দুইটি category-তে ভাগ করা যায়:

```
Primitive  
Reference / Non-Primitive
```

এই document-এ আমরা প্রথমে **Primitive Types** বিস্তারিতভাবে দেখব।

16. Primitive Data Types

JavaScript-এর primitive data types:

1. String
2. Number
3. Boolean
4. Undefined
5. Null
6. Symbol
7. BigInt

মনে রাখার জন্ম:

```
String
Number
Boolean
Undefined
Null
Symbol
BigInt
```

17. String

String হলো text বা character sequence।

```
const name = "Shariar";
```

Single quotes:

```
const name = 'Shariar';
```

Double quotes:

```
const name = "Shariar";
```

Template literals:

```
const name = `Shariar`;
```

তিনটিই string তৈরি করতে পারে।

17.1 String Example

```
const firstName = "Shariar";
const lastName = "Mahmud";

console.log(firstName);
console.log(lastName);
```

17.2 String Concatenation

```
const firstName = "Shariar";
const lastName = "Mahmud";

const fullName = firstName + " " + lastName;

console.log(fullName);
```

Output:

```
Shariar Mahmud
```

18. Template Literals

Modern JavaScript-এ dynamic string তৈরি করার জন্য template literals খুব useful।

```
const name = "Shariar";
const age = 31;

console.log(`My name is ${name} and I am ${age} years old.`);
```

Output:

```
My name is Shariar and I am 31 years old.
```

এখানে:

```
${name}
${age}
```

এর মধ্যে JavaScript expression লেখা যায়।

19. Number

JavaScript-এ integer এবং floating-point number একই `number` type-এর মধ্যে পড়ে।

```
const age = 31;  
    const price = 99.99;  
    const temperature = -5;
```

সবগুলো `number`।

```
typeof 31;
```

Output:

```
"number"
```

19.1 JavaScript Number-এর কিছু Special Value

Number type-এর মধ্যে কিছু special numeric values আছে:

```
Infinity  
-Infinity  
NaN
```

এগুলো আমরা পরের JavaScript section-এ বিস্তারিতভাবে দেখব।

20. Boolean

Boolean-এর মাত্র দুইটি value:

```
true  
false
```

Example:

```
const isLoggedIn = true;  
    const isAdmin = false;
```

Boolean সাধারণত decision-making-এর জন্য ব্যবহার হয়।

```
if (isLoggedIn) {  
  console.log("Welcome!");  
}
```

21. Undefined

কোনো variable declare করা হয়েছে কিন্তু value assign করা হয়নি, তাহলে তার value সাধারণত `undefined`।

```
let age;  
  
console.log(age);
```

Output:

```
undefined
```

এখানে variable আছে, কিন্তু value দেওয়া হয়নি।

21.1 Function-এর Return Value

কোনো function explicitly কিছু return না করলে তার return value `undefined` হয়।

```
function greet() {  
  console.log("Hello");  
}  
  
const result = greet();  
  
console.log(result);
```

Output:

```
Hello  
undefined
```

22. Null

`null` সাধারণত intentionally **empty** / **no value** বোঝাতে ব্যবহার করা হয়।

```
const selectedUser = null;
```

এর অর্থ:

এখন কোনো user selected নেই।

আরেকটি example:

```
let currentUser = null;

// পরে user login করলে
currentUser = {
  name: "Shariar"
};
```

23. `undefined` VS `null`

এটা interview-এ খুব common।

<code>undefined</code>	<code>null</code>
সাধারণত value দেওয়া হয়নি	ইচ্ছাকৃতভাবে empty value
JavaScript অনেক ক্ষেত্রে নিজে produce করে	Developer সাধারণত explicitly দেয়
<code>let x;</code> → undefined	<code>let x = null</code>
"value missing/not assigned"	"intentionally no value"

সহজ mental model:

```
undefined → value এখনো দেওয়া হয়নি
null       → এখানে ইচ্ছা করেই কোনো value নেই
```

24. Symbol

`Symbol` হলো একটি primitive type যা **unique identifiers** তৈরি করার জন্য ব্যবহার করা যায়।

```
const id1 = Symbol("id");
const id2 = Symbol("id");

console.log(id1 === id2);
```

Output:

```
false
```

দুটোর description একই হলেও দুইটি Symbol আলাদা।

24.1 Symbol কেন দরকার?

একটি object-এ এমন property তৈরি করতে চাইতে পারো যার key অন্য সাধারণ keys-এর সাথে conflict করবে না।

```
const userId = Symbol("userId");

const user = {
  name: "Shariar",
  [userId]: 123
};

console.log(user[userId]);
```

Symbol-এর practical use advanced JavaScript-এ বেশি দেখা যায়।

25. BigInt

JavaScript-এর `number` type-এর safe integer range সীমিত।

খুব বড় integer represent করার জন্য `BigInt` ব্যবহার করা যায়।

```
const bigNumber = 123456789012345678901234567890n;
```

শেষে:

```
n
```

দেওয়া হয়।

```
console.log(typeof bigNumber);
```

Output:

```
bigint
```

25.1 BigInt Example

```
const a = 12345678901234567890n;  
const b = 10n;  
  
console.log(a + b);
```

BigInt-এর সাথে সাধারণ Number সরাসরি arithmetic করা যায় না:

```
10n + 5;
```

এতে error হবে।

দুটোকেই একই numeric type করতে হবে:

```
10n + 5n;
```

26. Primitive vs Reference — Initial Mental Model

JavaScript data types বোঝার সময় একটি গুরুত্বপূর্ণ distinction:

Primitive Values

↓

String

Number

Boolean

Undefined

Null

Symbol

BigInt

আর অন্যদিকে:

Non-Primitive / Reference Values

↓

Object

Array

Function

...

Primitive values সাধারণত **immutable value** হিসেবে behave করে।

Example:

```
let name = "Shariar";
```

```
name = "Duke";
```

এখানে আগের string `"Shariar"` পরিবর্তন করা হয়নি; variable-এ নতুন value assign করা হয়েছে।

27. `typeof` Operator

কোনো value-এর type সম্পর্কে runtime-এ জানতে `typeof` operator ব্যবহার করা হয়।

Syntax:

```
typeof value
```

Example:

```
console.log(typeof "Hello");
```

Output:

```
string
```

28. `typeof` Examples

```
console.log(typeof "Shariar");  
// "string"  
  
console.log(typeof 31);  
// "number"  
  
console.log(typeof true);  
// "boolean"  
  
console.log(typeof undefined);  
// "undefined"  
  
console.log(typeof 123n);  
// "bigint"  
  
console.log(typeof Symbol("id"));  
// "symbol"
```

29. `typeof null` — Famous JavaScript Quirk

এটা JavaScript-এর খুব famous interview question।

```
console.log(typeof null);
```

Output:

```
"object"
```

কিন্তু `null` আসলে object নয়; এটি একটি primitive value।

এটা JavaScript-এর পুরোনো historical behavior/legacy quirk।

তাই:

```
typeof null === "object"
```

হলেও,

```
null ≠ object
```

30. `typeof` Function

Function-এর ক্ষেত্রে:

```
function greet() {  
  console.log("Hello");  
}  
  
console.log(typeof greet);
```

Output:

```
"function"
```

Technically JavaScript specification-এর type system-এ function object হলেও `typeof` তাদের জন্য বিশেষভাবে `"function"` return করে।

31. `typeof` দিয়ে Type Checking

Simple type checks করা যায়:

```
const age = 31;  
  
if (typeof age === "number") {  
  console.log("Age is a number");  
}
```

আর string:

```
const name = "Shariar";

if (typeof name === "string") {
  console.log("Name is a string");
}
```

32. Variable Naming Rules

Variable declaration শেখার সাথে naming rules-ও জানা দরকার।

Valid:

```
const name = "Shariar";
const userName = "Duke";
const age2 = 31;
const _value = 10;
const $price = 100;
```

Invalid:

```
const 2age = 31;
```

Variable name number দিয়ে শুরু করা যায় না।

আর:

```
const user-name = "Shariar";
```

এটিও valid variable name নয়।

33. JavaScript Case Sensitivity

JavaScript case-sensitive।

```
const name = "Shariar";
const Name = "Duke";
```

এগুলো দুইটি আলাদা identifier।

```
console.log(name);  
console.log(Name);
```

Output:

```
Shariar  
Duke
```

34. Naming Convention

JavaScript-এ সাধারণত **camelCase** ব্যবহার করা হয়।

Good:

```
const firstName = "Shariar";  
const userAge = 31;  
const isLoggedIn = true;  
const totalPrice = 500;
```

Avoid:

```
const firstname = "Shariar";  
const user_age = 31;
```

বিশেষ করে boolean-এর ক্ষেত্রে meaningful naming ভালো:

```
const isLoggedIn = true;  
const isAdmin = false;  
const hasPermission = true;
```

এতে code পড়তে সহজ হয়।

35. **const** মানেই Value Immutable নয়

এটা খুব important।

অনেকে ভাবে:

```
const
```

মানে value কখনো পরিবর্তন হতে পারবে না।

এটা পুরোপুরি সত্য নয়।

Primitive example:

```
const age = 31;  
  
// age = 32; ❌
```

কিন্তু object:

```
const user = {  
  name: "Shariar"  
};  
  
user.name = "Duke";  
  
console.log(user.name);
```

Output:

```
Duke
```

এখানে `user` variable-কে নতুন object assign করা হয়নি।

```
user = {};
```

এটা করা যাবে না।

কিন্তু existing object-এর property পরিবর্তন করা যায়।

এই distinction আমরা Objects chapter-এ আরও গভীরভাবে দেখব।

36. A Practical Example

ধরো একটি e-commerce application-এ user information রাখতে হবে:

```
const userName = "Shariar";  
const age = 31;  
const isLoggedIn = true;  
const email = "shariar@example.com";
```

এখানে:

```
userName    → string  
age         → number  
isLoggedIn  → boolean  
email       → string
```

তারপর:

```
console.log(`${userName} is ${age} years old.`);
```

Output:

```
Shariar is 31 years old.
```

37. Another Real-World Example

ধরো shopping cart:

```
const productName = "Keyboard";  
const price = 2500;  
let quantity = 1;  
const isAvailable = true;
```

Customer quantity পরিবর্তন করলে:

```
quantity = 2;
```

কিন্তু product name বা availability application-এর এই অংশে পরিবর্তন করার প্রয়োজন নেই:

```
const productName = "Keyboard";  
const isAvailable = true;
```

এখানে `const` এবং `let` ব্যবহারের practical difference দেখা যাচ্ছে।

38. Common Mistakes

Mistake 1 — সব জায়গায় `let`



```
let userName = "Shariar";
```

যদি value পরিবর্তন হওয়ার কোনো প্রয়োজন না থাকে, তাহলে:



```
const userName = "Shariar";
```

Mistake 2 — `const` reassign করা



```
const age = 31;
```

```
age = 32;
```

Mistake 3 — `const` initialization ছাড়া declare করা



```
const name;
```

Mistake 4 — `var` unnecessarily ব্যবহার করা

নতুন modern JavaScript code-এ সাধারণত:

```
const
let
```

দিয়ে কাজ করা ভালো।

Mistake 5 — `null` এবং `undefined` একই মনে করা

```
undefined
```

এবং:

```
null
```

এক জিনিস নয়।

Mistake 6 — `typeof null` দেখে ধরে নেওয়া `null object`

```
typeof null
```

এর output `"object"` হলেও `null` একটি primitive value।

39. Best Practices

Rule 1

Default হিসেবে:

```
const
```

ব্যবহার করো।

Rule 2

Value reassignment প্রয়োজন হলে:

```
let
```

ব্যবহার করো।

Rule 3

নতুন code-এ `var` সাধারণত avoid করো।

Rule 4

Meaningful variable names ব্যবহার করো।

```
const userAge = 31;
```

এটা:

```
const x = 31;
```

এর চেয়ে অনেক readable।

Rule 5

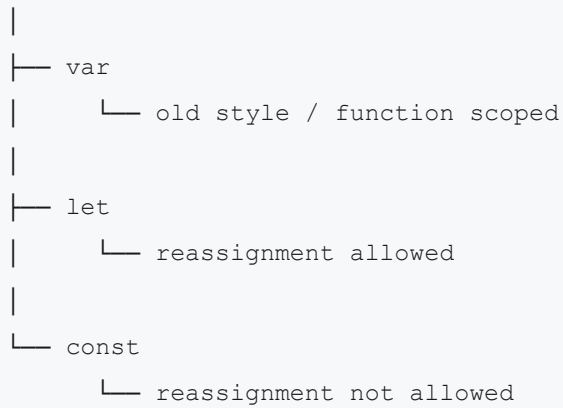
Boolean variable-এ `is`, `has`, `can`, `should` ইত্যাদি ব্যবহার করা useful।

```
const isActive = true;  
const hasPermission = false;  
const canEdit = true;
```

40. Mental Model

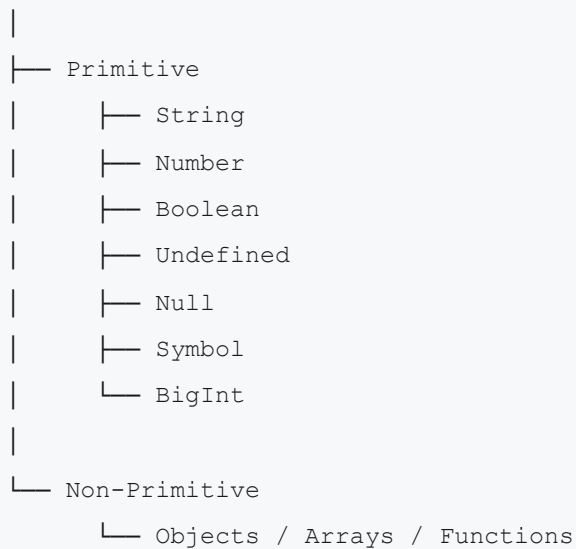
পুরো section-টা এভাবে মনে রাখতে পারো:

VARIABLE



আর:

DATA TYPE



আর type check:

```
typeof value
```



Exercises

এখন কিছু exercise। আগে নিজে solve করবো। সরাসরি answer দেখার চেষ্টা করবে না।

Exercise 1 — Variable Declaration

একটি program লেখো যেখানে:

- তোমার নাম
- বয়স
- দেশ
- তুমি coding করো কিনা

এই চারটি information variable-এ রাখবে।

তারপর সবগুলো `console.log()` করবে।

Exercise 2 — `let`

একটি `let` variable তৈরি করো:

```
score = 50
```

তারপর score পরিবর্তন করে:

```
80
```

করো এবং console-এ print করো।

Exercise 3 — `const`

একটি `const` variable তৈরি করো যেখানে তোমার জন্মের দেশ থাকবে।

তারপর সেটিকে অন্য country দিয়ে reassign করার চেষ্টা করো।

Question: কী হবে এবং কেন?

Exercise 4 — Data Types

নিচের প্রতিটির data type বের করো:

```
"Hello"  
42  
true  
undefined  
null  
123n  
Symbol("id")
```

`typeof` ব্যবহার করে output predict করো।

Exercise 5 — Real-world

একটি product-এর জন্য variables তৈরি করো:

```
name  
price  
quantity  
isAvailable
```

তারপর এমন একটি sentence console-এ দেখাও:

```
Keyboard costs 2500 taka and quantity is 2.
```



Output Prediction Questions

এগুলো নিজে output predict করার চেষ্টা করবে।

Question 1

```
var name = "Shariar";  
  
var name = "Duke";  
  
console.log(name);
```

Question 2

```
let age = 31;

age = 32;

console.log(age);
```

Question 3

```
const country = "Bangladesh";

country = "India";

console.log(country);
```

Question 4

```
let value;

console.log(value);

console.log(typeof value);
```

Question 5

```
console.log(typeof "100");

console.log(typeof 100);
```

Question 6

```
console.log(typeof null);
```

Extra question: এটা কেন `"object"` ?

Question 7

```
if (true) {  
    var name = "Shariar";  
}  
  
console.log(name);
```

Question 8

```
if (true) {  
    let name = "Shariar";  
}  
  
console.log(name);
```

Question: Question 7 এবং Question 8-এর behavior আলাদা কেন?

Interview Questions

নিজের ভাষায় answer দেওয়ার চেষ্টা করো।

1. `var`, `let`, এবং `const` -এর মধ্যে মূল পার্থক্য কী?
2. `let` কি reassign করা যায়?
3. `const` কি reassign করা যায়?
4. `const` declaration-এর সময় initialization কেন প্রয়োজন?
5. `var` এবং `let` -এর scope-এর মধ্যে পার্থক্য কী?
6. Block scope কী?
7. Function scope কী?
8. JavaScript-এর primitive data types কী কী?
9. `null` এবং `undefined` -এর পার্থক্য কী?
10. `typeof null` কেন `"object"` return করে?
11. Symbol কেন ব্যবহার করা হয়?
12. BigInt কখন ব্যবহার করা হয়?
13. `typeof` operator কী?
14. `const` কি object-এর property পরিবর্তন আটকায়?
15. Modern JavaScript-এ `var` কেন সাধারণত avoid করা হয়?

Challenge

একটি `user` information section তৈরি করো:

```
Name
Age
Email
Country
IsLoggedIn
```

Requirements:

- সঠিক variable declaration ব্যবহার করতে হবে।
- কোন value পরিবর্তন হবে আর কোনটা হবে না সেটা চিন্তা করে `let` / `const` ব্যবহার করতে হবে।
- প্রতিটি value-এর `typeof` console-এ দেখাতে হবে।
- একটি সুন্দর sentence তৈরি করতে হবে template literal দিয়ে।

Expected style:

```
Shariar is 31 years old and lives in Bangladesh.
Account status: Logged In
```

Quick Revision

Variables

```
const → default choice
let   → reassignment দরকার হলে
var   → old style, generally avoid
```

Primitive Types

```
string
number
boolean
undefined
null
symbol
bigint
```

Scope

```
var    → function scoped
let    → block scoped
const → block scoped
```

Type Checking

```
typeof value
```

Important Quirks

```
typeof null;
// "object"
```

`undefined` vs `null`

```
undefined → value assigned হয়নি / missing
null      → intentionally empty
```

Golden Rule

Use `const` by default. Use `let` when reassignment is necessary. Avoid `var` in modern JavaScript unless you have a specific reason to use it.